

Simulador de Canica sobre Superficie Topográfica

Proyecto Final de Programación – Ruta 1

Autor: Angel Andres Martinez Chavez

Mayo 2026

Resumen

Este proyecto implementa un simulador físico en Python que modela el movimiento de una partícula (canica) sobre diferentes superficies topográficas bidimensionales bajo la influencia de la gravedad y el rozamiento viscoso. Se resuelven numéricamente las ecuaciones de movimiento mediante integración Runge-Kutta (RK45) de SciPy, se calculan las energías cinética, potencial y total, y se generan visualizaciones gráficas de la trayectoria y la evolución energética. Además, se incluye un análisis avanzado que ajusta los parámetros de rozamiento y gravedad a partir de datos sintéticos con ruido utilizando `curve_fit`. El programa es interactivo por consola, permite previsualizar el terreno y la posición inicial, y está desarrollado siguiendo los principios de la programación orientada a objetos.

Requisitos de software y modelo de Python utilizado

El programa ha sido desarrollado y probado con:

- **Python 3.8 o superior** (se recomienda 3.8, 3.9, 3.10, 3.11 o 3.12).
- **Librerías necesarias** (con versiones mínimas):
 - `numpy` 1.20 o superior.
 - `scipy` 1.6 o superior.
 - `matplotlib` 3.3 o superior.

El código está escrito íntegramente en Python estándar, sin dependencias de entornos de notebook (no usa `ipywidgets`). Se ejecuta directamente en cualquier terminal o consola.

Índice

1. Introducción y objetivo	3
2. Física y matemática del problema	3
2.1. Ecuaciones de movimiento	3
2.2. Cálculo del gradiente numérico	4
2.3. Energías involucradas	4
2.4. Integración numérica	4

3. Terrenos disponibles	4
4. Arquitectura del código (orientación a objetos)	5
4.1. Clase abstracta <code>SistemaFisico</code>	5
4.2. Clase <code>Terreno</code>	5
4.3. Clase <code>Particula</code> (hereda de <code>SistemaFisico</code>)	5
4.4. Clases de eventos	5
4.5. Funciones auxiliares de entrada	6
4.6. Función de previsualización <code>previsualizar(...)</code>	6
4.7. Funciones de análisis y gráficas	6
4.8. Programa principal <code>main()</code>	6
5. Guía para el lector: ¿Qué debe hacer para usar el programa?	7
5.1. Pasos de instalación y ejecución	7
5.2. Flujo de interacción (qué aparecerá en pantalla)	7
5.3. ¿Qué esperar en las gráficas?	8
5.4. Ejemplo de salida en consola	8
6. Análisis avanzado: ajuste de parámetros con <code>curve_fit</code>	8
7. Posibles ampliaciones futuras	9
8. Conclusiones	9

1. Introducción y objetivo

El objetivo de este proyecto es diseñar una aplicación en Python que modele, simule y analice el comportamiento de un sistema físico real: el movimiento de una canica sobre un terreno irregular bajo los efectos de la gravedad y el rozamiento.

El programa permite al usuario:

- Elegir entre cinco terrenos predefinidos.
- Fijar la posición inicial, velocidad inicial, coeficiente de rozamiento y gravedad.
- Previsualizar el terreno con la posición elegida antes de la simulación.
- Ejecutar la simulación hasta que la canica se detenga o salga del dominio $[-7, 7] \times [-7, 7]$.
- Visualizar la trayectoria sobre un mapa topográfico y la evolución temporal de las energías.
- Realizar un análisis avanzado con ajuste de parámetros (`curve_fit`) utilizando datos ruidosos sintéticos.

Todo el código sigue los principios de la programación orientada a objetos: clases, herencia, encapsulación, y una clase base abstracta.

2. Física y matemática del problema

2.1. Ecuaciones de movimiento

La partícula se mueve en el plano xy sobre una superficie cuya altura es $z = h(x, y)$. La fuerza gravitatoria que actúa sobre la partícula es $m\mathbf{g}$, pero solo la componente tangencial a la superficie acelera la partícula. Para una superficie $z = h(x, y)$, la pendiente en las direcciones x e y viene dada por el gradiente $\nabla h = \left(\frac{\partial h}{\partial x}, \frac{\partial h}{\partial y}\right)$. La aceleración debida a la gravedad es:

$$\mathbf{a}_{\text{grav}} = -g\nabla h.$$

Además, se considera un rozamiento viscoso lineal: $\mathbf{F}_{\text{roz}} = -m\mu\mathbf{v}$. Por lo tanto, la aceleración total (por unidad de masa) es:

$$\frac{d^2x}{dt^2} = -g\frac{\partial h}{\partial x} - \mu\frac{dx}{dt}, \quad \frac{d^2y}{dt^2} = -g\frac{\partial h}{\partial y} - \mu\frac{dy}{dt}.$$

Escribiendo el sistema de segundo orden como un sistema de primer orden (con $v_x = \dot{x}$, $v_y = \dot{y}$):

$$\begin{cases} \dot{x} = v_x, \\ \dot{y} = v_y, \\ \dot{v}_x = -g\frac{\partial h}{\partial x} - \mu v_x, \\ \dot{v}_y = -g\frac{\partial h}{\partial y} - \mu v_y. \end{cases}$$

2.2. Cálculo del gradiente numérico

Para no tener que derivar analíticamente las funciones de los terrenos, se calcula el gradiente mediante diferencias centradas con un paso pequeño $h = 10^{-5}$:

$$\frac{\partial h}{\partial x}(x, y) \approx \frac{h(x + \Delta, y) - h(x - \Delta, y)}{2\Delta}, \quad \frac{\partial h}{\partial y}(x, y) \approx \frac{h(x, y + \Delta) - h(x, y - \Delta)}{2\Delta}.$$

2.3. Energías involucradas

- **Energía cinética:** $E_c = \frac{1}{2}m(v_x^2 + v_y^2)$.
- **Energía potencial gravitatoria:** $E_p = mg h(x, y)$.
- **Energía mecánica total:** $E_T = E_c + E_p$.

Debido al rozamiento, la energía total disminuye con el tiempo (sistema disipativo). La masa m se cancela en las ecuaciones de movimiento, por lo que la trayectoria no depende de ella; sin embargo, se mantiene en los cálculos energéticos para mostrar valores en julios (se fija $m = 1$ kg).

2.4. Integración numérica

El sistema de ecuaciones diferenciales ordinarias (EDO) se resuelve con el método explícito de Runge-Kutta de orden 4-5 (RK45), implementado en la función `solve_ivp` de SciPy. Se definen dos eventos para detener la integración automáticamente:

- **PararPorVelocidad:** se activa cuando la rapidez $\sqrt{v_x^2 + v_y^2}$ cae por debajo de un umbral (por defecto 0,005 m/s). Indica que la canica se ha detenido.
- **LimiteDominio:** se activa cuando $\max(|x|, |y|) \geq 7,0$, es decir, la partícula abandona el dominio simulado. Esto evita cálculos fuera de rango.

3. Terrenos disponibles

El diccionario `TERRENOS` contiene cinco superficies predefinidas. Cada una se implementa como una función lambda que recibe x e y y devuelve la altura z . Las funciones son:

Nombre	Función $z = h(x, y)$
Montaña clásica	$\sin(0,8x) \cos(0,8y) + 0,5 \sin(1,5x) \cos(1,2y) + 0,3 \exp(-(x^2 + y^2)/4)$
Plano inclinado + montículos	$-0,6x + 0,8 \exp(-((x-1)^2 + (y-1)^2)/1,2) + 0,6 \exp(-((x+0,5)^2 + (y+1)^2)/1,0) - 0,5 \exp(-((x+1)^2 + (y-1)^2)/1,5)$
Dos picos y valle central	$1,2 \exp(-((x+1,5)^2 + (y+1,5)^2)/1,5) + 1,2 \exp(-((x-1,5)^2 + (y-1,5)^2)/1,5) - 0,8 \exp(-(x^2 + y^2)/2) + 0,1 \sin(1,2x) \cos(1,2y)$
Silla de montar	$0,8(x^2 - y^2) \exp(-(x^2 + y^2)/6) - 0,5 \exp(-((x-1)^2 + y^2)/2)$

$$\text{Montaña rugosa} \quad \sin(0,8x) \cos(0,8y) + 0,5 \sin(1,5x) \cos(1,2y) + 0,3 \sin(2,2x) \cos(2,0y) + 0,2 \sin(3,0x) \cos(2,5y) + 0,3 \exp(-(x^2 + y^2)/4)$$

Todas las funciones están definidas para $x, y \in [-7, 7]$, que es el dominio de simulación.

4. Arquitectura del código (orientación a objetos)

A continuación se describen las principales clases y su responsabilidad.

4.1. Clase abstracta SistemaFisico

Define la interfaz común para cualquier sistema dinámico. Obliga a las clases hijas a implementar los métodos `derivada` y `simular`.

```

1 class SistemaFisico(ABC):
2     @abstractmethod
3     def derivada(self, t, estado):
4         pass
5     @abstractmethod
6     def simular(self, t_span, estado_inicial, t_eval=None):
7         pass

```

4.2. Clase Terreno

Almacena la función de altura y proporciona métodos para:

- Calcular el gradiente numérico.
- Calcular la energía potencial en un punto.

4.3. Clase Particula (hereda de SistemaFisico)

Es el corazón de la simulación. Contiene:

- Atributos: el terreno, la masa (aunque no influye en la trayectoria), rozamiento, gravedad, umbral de velocidad y límite del dominio.
- `derivada(t, estado)`: implementa las ecuaciones del movimiento.
- `simular(...)`: crea los eventos y llama a `solve_ivp`.

4.4. Clases de eventos

- `PararPorVelocidad`: detiene la integración cuando la rapidez baja del umbral.
- `LimiteDominio`: detiene cuando la partícula sale del cuadrado $[-7, 7] \times [-7, 7]$.

Ambas clases tienen un método `__call__` que se evalúa en cada paso de integración y devuelven una función que se anula en el evento. Los atributos `terminal=True` y `direction` controlan el comportamiento.

4.5. Funciones auxiliares de entrada

- `input_float(mensaje, min_val, max_val)`: pide un número real validando el rango.
- `input_posicion(mensaje, limite)`: llama a `input_float` con límites ± 7 .
- `input_velocidad(mensaje, max_vel)`: límite ± 20 m/s.
- `velocidad_maxima_recomendada(...)`: estima la velocidad máxima segura para no salir del dominio basándose en la diferencia de altura máxima posible.

4.6. Función de previsualización `previsualizar(...)`

Dibuja el mapa topográfico (contourf) y superpone un punto rojo en la posición elegida. Utiliza `plt.show(block=False)` para que la ventana no bloquee la consola, y luego espera a que el usuario presione Enter para cerrar la figura y continuar.

4.7. Funciones de análisis y gráficas

- `calcular_estadisticas(...)`: imprime tiempo total, posiciones, velocidades, distancia recorrida (integral trapezoidal de la rapidez), energías inicial/final y pérdida.
- `graficar_trayectoria(...)`: muestra el mapa de contorno y la trayectoria completa (línea roja, punto verde inicio, punto azul fin).
- `graficar_energias(...)`: dibuja la evolución de E_c , E_p y E_T frente al tiempo.
- `analisis_curva_fit(...)`: genera datos sintéticos ruidosos a partir de una simulación con parámetros “reales” y utiliza `curve_fit` para recuperar rozamiento y gravedad; calcula MSE y MAE.

4.8. Programa principal `main()`

Controla el flujo completo:

1. Bucle de configuración: permite elegir terreno, posición, previsualizar y decidir entre continuar, cambiar terreno o cambiar posición.
2. Solicitud de velocidad, rozamiento y gravedad.
3. Ejecución de la simulación.
4. Presentación de resultados (estadísticas y gráficas).
5. Ejecución automática del análisis `curve_fit`.

5. Guía para el lector: ¿Qué debe hacer para usar el programa?

5.1. Pasos de instalación y ejecución

1. Asegurarse de la compatibilidad de versión para esta simulación se usó Python 3.14.3
2. Guarde el código completo en un archivo `simulador.py`.
3. Instale las dependencias (si no las tiene):

```
pip install numpy scipy matplotlib
```

4. Ejecute el programa en una terminal o consola:

```
python simulador.py
```

5.2. Flujo de interacción (qué aparecerá en pantalla)

Al ejecutar, el programa mostrará mensajes en español y esperará entradas del usuario.

1. **Selección del terreno:** Se listan 5 opciones. Introduzca el número correspondiente.
2. **Posición inicial:** Ingrese coordenadas x e y dentro de $[-7, 7]$.
3. **Previsualización:** Se abrirá una ventana con el mapa y un punto rojo. En la consola aparece:
Presiona Enter después de ver el mapa para continuar...
Observe el mapa, luego presione **Enter** (la ventana se cerrará automáticamente).
4. **Menú de opciones:**

¿Qué desea hacer?

1. Continuar con la simulación (usar este terreno y posición)
2. Cambiar de terreno (elegir otro mapa)
3. Cambiar solo la posición (mantener este terreno)

Elija 1 para continuar, 2 o 3 para repetir la previsualización con nuevos valores.

5. **Velocidad inicial:** Se mostrará la velocidad máxima recomendada. Introduzca v_x y v_y (puede ser mayor a la recomendada, pero la canica podría salirse).
6. **Rozamiento y gravedad:** Ingrese μ (0–1) y g (2–20 m/s²).
7. **Simulación:** Mientras se ejecuta, la consola muestra información del progreso. Al terminar, aparecen:
 - Estadísticas detalladas.
 - Dos ventanas de gráficos (pueden superponerse; ciérrelas para terminar el programa).
8. **Análisis avanzado:** Se ejecuta automáticamente al final y muestra en consola los resultados del ajuste con `curve_fit`.

5.3. ¿Qué esperar en las gráficas?

- **Trayectoria:** mapa de colores que representa la altura. La trayectoria de la canica se dibuja en rojo; el punto de inicio es verde y el punto final azul.
- **Energías:** tres curvas frente al tiempo: cinética (naranja), potencial (azul) y total (negra discontinua). La energía total decrece debido al rozamiento.

5.4. Ejemplo de salida en consola

```
=====
ANÁLISIS DE LA SIMULACIÓN
=====
Tiempo total: 8.24 s
Posición inicial: (2.00, 2.00)
Posición final:   (-1.23, 1.45)
Velocidad inicial: (4.00, 4.00) m/s -> rapidez: 5.657 m/s
Velocidad final:   (-0.00, 0.00) m/s -> rapidez: 0.000 m/s
Rapidez media: 1.234 m/s
Aceleración media: (-0.48, -0.48) m/s²
Distancia recorrida: 10.17 m

Energía inicial: Cinética=16.00 J, Potencial=29.43 J, Total=45.43 J
Energía final:   Cinética=0.00 J, Potencial=20.12 J, Total=20.12 J
Pérdida de energía: 25.31 J
La canica se detuvo por rozamiento.
=====
```

6. Análisis avanzado: ajuste de parámetros con `curve_fit`

Para demostrar el uso de técnicas de optimización y análisis de datos, el programa incluye un módulo que:

1. Toma el terreno actual y fija unos valores “reales” de rozamiento ($\mu_{\text{real}} = 0,12$) y gravedad ($g_{\text{real}} = 9,8$).
2. Simula la trayectoria sin ruido y genera 150 puntos de posición.
3. Añade ruido gaussiano (desviación típica 0.03) a esas posiciones, simulando mediciones experimentales.
4. Define una función `modelo_ajuste(t, roz, g)` que devuelve la trayectoria simulada para esos parámetros.
5. Utiliza `scipy.optimize.curve_fit` para encontrar los valores $\hat{\mu}$ y $\hat{g}$ que minimizan la suma de cuadrados de los residuos.
6. Muestra los valores ajustados con sus incertidumbres, y los errores MSE y MAE.

Este análisis no afecta la simulación principal, pero es parte esencial del proyecto para demostrar el uso de SciPy en tareas de inferencia de parámetros.

7. Posibles ampliaciones futuras

- Añadir una animación de la canica moviéndose.
- Permitir la exportación de los datos de la simulación a un archivo CSV.
- Incorporar una interfaz gráfica (Tkinter, PyQt) para evitar la consola.
- Cargar terrenos desde archivos externos (matrices de altura).
- Implementar rebotes con coeficiente de restitución.
- Paralelizar múltiples simulaciones para barridos de parámetros.

8. Conclusiones

Se ha desarrollado un simulador completo y funcional que integra conceptos de física (gravedad, rozamiento, energías), métodos numéricos (integración de EDOs, diferenciación numérica, ajuste de curvas) y buenas prácticas de programación (orientación a objetos, herencia, abstracción). La interfaz por consola es intuitiva y permite al usuario explorar diferentes configuraciones antes de la simulación final. Los resultados se presentan de forma clara, tanto en texto como en gráficos, facilitando la interpretación del comportamiento del sistema.

El programa cumple con todos los requisitos exigidos en la Ruta 1 del proyecto final, demostrando el dominio de Python, las librerías científicas y la capacidad de plantear y resolver un problema de simulación física.